

PhD Candidate Interview Questions: AI-Agent Development for Energy Systems

Note!

- You may choose 1 of the following 2 questions to answer. If you feel capable, you are welcomed to answer both questions.
 - For each question, part 1 is mandatory, part 2 is optional.
 - It is allowed to use AI tools to assist writing and coding, but we prioritize original reasoning, reproducible design, and engineering reliability. Content that is generic, untested, or not grounded in realistic energy-system constraints will be ranked low.
 - You should prepare in English.
-

Question 1: Strategic AI-agent for harbor electrification planning

Background

A Swedish harbor is undergoing an ambitious energy transition toward full electrification and decarbonization. The transition involves multiple interconnected sectors:

- **Industrial processes:** Manufacturing, logistics operations, and port equipment requiring electrification
- **Transportation:** Heavy goods vehicles, port machinery, and potentially shore power for vessels

Decision-makers face complex challenges in developing a reliable and executable roadmap, including technology selection, system sizing, economic viability assessment, and navigating social and regulatory considerations. The time scope is 10 years.

Task

Design and develop a conceptual **LLM-based AI-agent** that can support strategic decision-making for harbor electrification. The decisions mainly concern investments (economy) and system operation and management after technology installations. The agent should be able to:

- Reason over technology choices (e.g., heat pumps, battery energy storage system, microgrids, shore-side charging),
- Query structured energy system simulation outputs for feasibility,
- Generate explainable and uncertainty-aware transition recommendations.

Your submission should include:

Part A: Architecture Design (mandatory, 1 page summary)

1. **Agent Architecture:** Describe the overall architecture of your AI-agent. Consider:
 - What type of agent architecture would you use (e.g., ReAct, Plan-and-Execute, multi-agent system, or any other choices)?
 - How would you structure the interaction between the LLM and external tools/data sources?
 - How would the agent handle multi-step reasoning for complex planning tasks?
2. **Data Sources & Tools:** Identify and list which kinds of data are needed and describe the required features of the data the agent should access
3. **Transparency & Explainability:** Describe methods to ensure:
 - The agent's recommendations are explainable to non-technical stakeholders
 - Uncertainty in predictions and recommendations is communicated appropriately
 - Decision provenance can be traced
4. **Reliability & Testing:** Propose a strategy for:
 - Validating agent outputs against expert knowledge
 - Handling hallucinations and ensuring factual grounding
 - Testing edge cases and failure modes

Part B*: Implementation Prototype (optional, Python Code with necessary explanation)

In this part, you shall **provide the AI-Agent development pipeline** in Python (model selection, orchestration framework, prompt engineering, reliability layers, monitoring, failure handling). Provide sample Python code demonstrating key components of your agent. Your code should include:

1. A basic agent structure using an LLM framework (e.g., LangChain, LlamaIndex, or direct API calls)
2. At least **two custom tools** or **API access to existing open-source tools** relevant to energy decision-making (e.g., a simplified load estimation tool, a technology cost calculator, a CO₂ emission estimator, or other modelling packages)
3. A demonstration of how the agent would handle a sample query such as: *"What renewable energy mix would you recommend for the harbor's building sector, considering a 10-year payback requirement?"*

Note: You may use placeholder data or simplified models. The focus is on demonstrating understanding of agent architecture and tool integration, not production-ready code.

Question 2: Operational AI-agent for energy community BESS management

Background

A Swedish industrial area is transforming into an **energy community** where locally generated solar PV energy can be shared among participants. To address the temporal mismatch between PV generation and electricity consumption, and to respect grid import/export limits, a shared **Battery Energy Storage System (BESS)** will be installed.

The community includes:

- Office buildings with typical commercial load patterns
- Logistics centers with variable loads depending on operations
- Light manufacturing facilities
- EV charging stations (passenger vehicles and heavy goods vehicles)

Stakeholders need an intelligent system to advise on BESS operation strategies that maximize collective benefits while respecting technical constraints.

Task

Design and develop a conceptual **LLM-based AI-agent** for operational BESS management advisory. Assume the Agent can shift load according to local demand and grid limits, consider future EV and heavy goods charging profiles, and return operational recommendations with explanations and uncertainty indicators.

Your submission should include:

Part A: Architecture Design (mandatory, 1 page summary)

1. **Agent Architecture:** Describe your proposed agent design:
 - How would the agent process real-time and forecast data?
 - What is the interaction flow between the user (stakeholder), the LLM, and backend systems?
 - How would you handle the need for both immediate responses and computationally intensive optimization?
2. **Data Integration:** Identify and list which kinds of data are needed and describe the required features of the data the agent should access
3. **Decision Support Logic:** Explain how the agent would:
 - Translate natural language queries into actionable recommendations

- Interface with optimization algorithms or rule-based systems
 - Balance multiple objectives (cost minimization, self-consumption maximization, grid support)
4. **Uncertainty Handling:** Describe approaches for:
- Communicating forecast uncertainty to users
 - Robust decision-making under uncertain PV/load forecasts
 - Learning from prediction errors over time

Part B*: Implementation Prototype (optional, Python Code with necessary explanation)

Provide **agent development pipeline** as well as the sample Python code demonstrating:

1. An agent capable of answering operational questions about BESS management
2. At least **two custom tools** or **API access to existing open-source tools**, for example:
 - A BESS state simulator (simplified model of charge/discharge dynamics)
 - A self-consumption calculator
 - An electricity cost estimator based on time-of-use pricing
3. Integration of (simplified or mock) time-series data for PV generation and load
4. A demonstration handling queries such as:
 - *"Should we charge or discharge the battery in the next 2 hours given the current conditions?"*
 - *"What was our self-consumption rate yesterday and how could it be improved?"*

Note: Simplified models and synthetic data are acceptable. Focus on demonstrating the integration of LLM reasoning with domain-specific calculations.

Submission Guidelines

- **Format:** PDF document containing written responses. Code demonstration may be submitted as separate .py files or a link to GitHub repository.
 - **Deadline:** 2026-08-24 Monday at 23:59
 - **Resources:** Candidates may use any publicly available resources, documentation, and AI assistants, but must be prepared to explain and discuss their submissions in a follow-up interview
 - **Send your answers to** chang.su@mdu.se
-